ITPROFEL2: Open Source Programming

Learning Module: Finals - Week 7

Instructor: Henry V. Dela Cruz | *Topic: Constructors and Instance Variables*

I. Overview and Objectives

Last week, we assigned instance variables using a regular method. However, in professional development, an object should be fully initialized and ready to use the exact moment it is spawned. We guarantee this using **Constructors**.

> **Intended Learning Outcomes (ILOs):**
>
> - Differentiate between defining instance variables in normal methods versus constructors.
> - Implement the `__init__` constructor to enforce data requirements upon instantiation.
> - Initialize complex instance variables using imported logic modules.

II. The Flaw of Normal Methods

If you define instance variables inside a standard method, a critical error can occur if another developer tries to access that data *before* the setup method is called.

```
class Profile:
    def setup(self, name):
        self.username = name

p = Profile()
# print(p.username) <-- CRASH! AttributeError. 'setup()' was never executed.
```

III. The Constructor: __init__

A **Constructor** is an automatic initialization method. In Python, it is strictly named `__init__`. The absolute millisecond you instantiate a class, Python looks for the `__init__` method and runs it automatically. It is the perfect place to set up mandatory instance variables.

```
class Profile:
    # The constructor demands arguments right when the object is created
    def __init__(self, name, age):
        self.username = name
        self.user_age = age

# We pass the arguments directly into the class parentheses during instantiation
user_obj = Profile("Henry", 21)

print(user_obj.username) # Output: Henry (Safe! Data was guaranteed at birth)
```

Laboratory Session: Dynamic Entity Initialization

Scenario: It is time to complete the core physics of your `UserEntity` architecture. You must rewrite the class to use an `__init__` constructor. Upon creation, the object must automatically demand a user's raw name and year, clean the data, generate a random ID, and lock these into its own instance variables.

Instructions:

1. Take your `UserEntity` class. Ensure you `import random` at the top of your script.
2. Define the constructor: `def __init__(self, raw_name, reg_year):`
3. Inside the constructor, create an instance variable: `self.name = raw_name.title().strip()`. Notice how we are reusing our string cleaning logic from Week 2!
4. Inside the constructor, create a second instance variable: `self.system_id`. Generate this by combining the `reg_year` parameter with a `random.randint(1000, 9999)`. This is our Week 1 logic!
5. Outside the class, instantiate the object directly: `final_user = UserEntity(" hEnry deLA cRuz ", 2026)`.
6. Print the object's cleaned `.name` and generated `.system_id` attributes to prove the constructor autonomously cleaned and randomized the data.

Expected Terminal Output:

```
--- ENTITY AUTO-INITIALIZATION ---
Constructing object...

Entity Name: Henry Dela Cruz
Assigned System ID: 2026-8492
Status: Initialization Successful.
```

Submit your `.py` *file. Your isolated stealth components are fully operational. Prepare for next week.*